Herald

Herald — Integration Guide

Field	Value
Revision	1
Created	2026-05-22
Last modified	2026-05-22
Status	active
Status summary	First-cut consumer integration guide written after Wave 3b lands the §32 7-stage Runner (commit `c2b67c3`). Covers: adding Herald as a submodule, the parent-discovery contract for the HelixConstitution, credential configuration via the wizard, running migrations, starting `pherald serve`, emitting a first CloudEvent via `POST /v1/events`, verifying delivery, running the anti-bluff battery. Known limitations + Wave 4 transport roadmap (HTTP/3 + Brotli + TOON — design doc landed in commit `c60b3fd`) called out at the end.
Issues	none
Issues summary	—
Fixed	(n/a — first revision)
Continuation	Bump revisions as Wave 4a (HTTP/3 + Brotli) and Wave 4b (TOON) land; the consumer-facing API changes there require integration-guide refresh.

Table of contents

---

§1. What Herald is

Herald is a multi-tenant, multi-channel **notification fan-out platform** built on Go 1.25+, Postgres, and Redis. The mission (from spec V3 §1):

> Ingesting system events and reliably fanning them out to multiple notification channels so every alert reaches the right destination without confusion.

Inbound events use the **CloudEvents v1.0** envelope. Outbound delivery flows through pluggable **channel adapters** (today: `null://` sandbox + Telegram; coming: Slack, Email, Max, Discord, Teams, Lark, WhatsApp, Viber, ntfy, Gotify, webhook). All deliveries are recorded in `outbound_delivery_evidence` so the operator has a forensic trail.

Herald ships as **seven flavor binaries** (Wave 2 r1):

Binary	Default port	What it does
`pherald`	24791	**Project Herald** — the event ingest path. `POST /v1/events` accepts CloudEvents and routes them through the §32 7-stage Runner (parse → idempotency → tenant → policy → subscriber fan-out → channel dispatch → outcome record).
`sherald`	24793	**System Herald** — host-safety daemon. `GET /v1/safety_state` returns process-local counters (open events, mem%, last destructive-op log). §43 commands wrap `rm` / `git-reset` / `git-push-force` with prerequisite checks.
`cherald`	24792	**Constitution Herald** — policy evaluator. `GET /v1/compliance` returns paginated `constitution_state` rows (the audit trail of every policy decision). §43 commands include creds-scan, docs-sync, composite-gate.
`iherald`	24794	**Incident Herald** — credential-leak page-out + operator-blocked escalation. `POST /v1/webhooks/page` (currently a 501 stub awaiting HRD-024).
`bherald`	— (CLI-only)	**Build Herald** — CI/test bindings: evidence-capture, test-tier-verify, gate-retest.
`rherald`	— (CLI-only)	**Release Herald** — tag-mirror, changelog-generate, gate-retest.
`scherald`	— (CLI-only)	**Scheduled-audit Herald** — periodic Status.md sweep + compliance digest.

The seven binaries consume the shared `commons/cli/` scaffold (Wave 2) and `commons_auth/` JWT middleware (Wave 3a). Each `cmd/<flavor>/main.go` is ~25 LOC.

---

§2. Prerequisites

You will need:

Component	Version	Why
Go	1.25.0 or newer (1.26 verified)	Workspace + race-detector + `runtime.MemStats`
Postgres	15+	`events_processed`, `subscribers`, `subscriber_aliases`, `outbound_delivery_evidence`, `constitution_state`, `constitution_bindings`, `claude_code_sessions` tables — see migrations `000001..000012` under `commons_storage/migrations/`. RLS-enforced; uses `app.current_tenant_id` GUC.
Redis	7+	Hot idempotency cache (24h SETNX TTL by default) + JWKS cache (5min default).
Container runtime	`podman` or `docker`	For the quickstart Compose stack + e2e_bluff_hunt invariants E13-E18. Optional if you bring your own PG/Redis.
Python 3.9+	(host)	Used by the e2e harness + the doc-export pipeline.
pandoc + weasyprint	(host)	For regenerating `.html`/`.pdf`/`.docx` sibling artefacts. Optional unless you're contributing to docs.
`claude` CLI	Optional	Required only for the Claude Code dispatcher (HRD-012); not needed for plain channel fan-out.

For development you also want:

- `gh` CLI (for catalogue-checks per §11.4.74)
- `psql` (manual schema inspection)

---

§3. Adding Herald to a parent project (as a submodule)

```
# In the parent project's root:
git submodule add git@github.com:vasic-digital/Herald.git
        submodules/herald
git submodule update --init --recursive submodules/herald
```

Herald is a self-contained Go workspace (`go.work` listing 14 modules: 7 foundation + `commons_auth` + 6 flavor binaries). Per spec §9.1 `go.work` is **gitignored** — you'll need to initialize it on a fresh clone:

```
cd submodules/herald
go work init
go work use ./commons ./commons_auth ./commons_constitution ./
        commons_infra \
            ./commons_messaging ./commons_prefix ./
        commons_storage \
            ./pherald ./sherald ./cherald ./bherald ./rherald ./
        iherald ./scherald
```

(Or check `go.work.example` if Herald ships one — the workspace shape is stable post-Wave-3a.)

Verify the build works:

```
go build ./commons/... ./commons_prefix/... ./
        commons_messaging/... \
         ./commons_storage/... ./commons_constitution/... ./
        commons_infra/... \
         ./pherald/... ./sherald/... ./cherald/... ./
        bherald/... ./rherald/... \
         ./iherald/... ./scherald/...
```

Silent output = clean.

---

§4. The parent-discovery contract

Herald is consumed as a submodule of a parent project that already carries the **Helix Constitution** submodule at `<parent>/constitution/`. Herald therefore does NOT keep its own copy. Locate the constitution from any nested depth by walking up:

```
CONST_DIR="$(bash "$(find . -type d -name constitution -print
        -quit 2>/dev/null)/find_constitution.sh")"
```

Or, more robustly, from any starting directory:

```
CONST_DIR="$(bash <ancestor>/constitution/find_constitution.sh)"
```

For standalone development of Herald (no parent project), clone the constitution alongside Herald:

```
git clone git@github.com:HelixDevelopment/HelixConstitution.git \
    $(dirname "$PWD")/constitution
```

Once located, all rules in `<discovered>/Constitution.md`, `CLAUDE.md`, `AGENTS.md`, and `QWEN.md` apply unconditionally. Herald's `docs/guides/HERALD_CONSTITUTION.md` extends them — it MUST NOT weaken any inherited rule.

The inheritance gate `tests/test_constitution_inheritance.sh` (15 invariants; I8a/b/c assert the §107 anti-bluff covenant anchor is present in CLAUDE.md / AGENTS.md / HERALD_CONSTITUTION.md) MUST pass before any commit touching root docs.

---

§5. Configuring credentials (the wizard)

`pherald wizard credentials [service]` is the canonical credential-setup tool. Supports interactive AND non-interactive (CI-friendly) modes:

Interactive (the operator types things):

```
pherald wizard credentials telegram
```

Non-interactive (flag-driven):

```
pherald wizard credentials telegram \
    --bot-token=8000000000:XXXXXXX \
    --chat-id=987654321 \
    --shell-target=zshrc \
    --non-interactive
```

Env-driven (most ergonomic for operators who already exported the vars):

```
export HERALD_TGRAM_BOT_TOKEN=...   # already in your .zshrc?
pherald wizard credentials telegram   # auto-detects, validates
        via getMe, prompts only for chat_id
```

The wizard:

1. Resolves each value by **flag → env → prompt** order.
2. Validates EVERY value against its source-of-truth API BEFORE persisting (Telegram `getMe` for tokens, `getChat` for chat IDs, `claude --version` for the dispatcher binary). A token that "looks right" but fails the API call is rejected at the prompt, not silently saved.
3. Appends `export FOO=bar` lines to your shell-startup file (`~/.zshrc` by default).
4. Mirrors a MASKED summary to `~/.herald/credentials.md` (chmod 600, git-ignored).
5. Never writes raw secrets to anything tracked by git.

§107 evidence baked in: every step that claims success carries positive runtime evidence. The wizard's smoke-test invariants (see `pherald/internal/wizard/telegram_test.go`) prove against an httptest fixture that opts/env/prompt all work as documented.

Available services: `telegram`, `claude-code`, `all`. Other channels (Slack, Email, etc.) are documented in `docs/guides/messengers/` but their wizard flows haven't shipped yet.

---

§6. Database + Redis (migrations + bring-up)

The quickest path is the bundled compose stack:

```
# From Herald root:
podman-compose -f quickstart/docker-compose.quickstart.yml --
        project-name herald-mvp up -d
```

This brings up:

- Postgres on `127.0.0.1:24100` (per spec §27 — port 70xxx range; default DB `herald`, user `herald`, password from `HERALD_DB_PASSWORD` env)
- Redis on `127.0.0.1:24200` (password from `HERALD_REDIS_PASSWORD` env)
- OTel collector on `127.0.0.1:24300` (optional — observability stack)

Apply migrations:

```
export HERALD_PG_DSN="postgres://herald:<your-
        password>@127.0.0.1:24100/herald"
pherald migrate up
pherald migrate status   # expect: schema version: 12
```

Schema currently includes:

- `tenants`, `subscribers`, `subscriber_aliases`, `agent_tokens` (§7)
- `channel_addresses` (§6)
- `outbound_delivery_evidence` (§16) — audit trail
- `events_processed` (§32.2) — inbound idempotency archive
- `constitution_state`, `constitution_bindings` (§42) — policy decisions + per-binding mode ladder
- `claude_code_sessions` (§33.2) — Claude Code dispatcher session persistence
- `quarantined_messages`, `webhook_sources`, `thread_refs`, `background_tasks` — supporting tables

All non-trivial tables enable **FORCE ROW LEVEL SECURITY** with a policy keyed on `app.current_tenant_id`. Reads + writes MUST happen inside `commons_storage.WithTenantContext(ctx, tenantID)` — bypassing it is impossible by construction (Postgres FORCE RLS applies even to the table owner).

---

§7. Starting `pherald serve`

`pherald serve` is the live HTTP ingest daemon. It needs:

```
# Required:
export HERALD_PG_DSN="postgres://
        herald:<password>@127.0.0.1:24100/herald"
export HERALD_AUTH_MODE=hmac                    # or "jwks"
export HERALD_AUTH_HMAC_SECRET=<32+ random bytes>

# Optional:
export HERALD_REDIS_URL="redis://:<password>@127.0.0.1:24200/0"
        # enables hot idempotency cache
export HERALD_TGRAM_BOT_TOKEN=...               # registers
        Telegram channel
export HERALD_TGRAM_CHAT_ID=...                 # not strictly
        needed at server start (per-subscriber)
```

Then:

```
pherald serve --http-port 24791
```

You should see:

- `/v1/healthz` and `/v1/readyz` and `/metrics` come up immediately (bypass JWT; K8s-probe friendly)
- `POST /v1/events` is JWT-gated — every request needs `Authorization: Bearer <jwt>` with `tenant` and `sub` claims

Press `Ctrl-C` (SIGTERM) → graceful shutdown drains in-flight requests + closes PG/Redis pools.

---

§8. Emitting your first CloudEvent

Generate a JWT for your tenant (HMAC mode example — for JWKS, your IdP issues the token):

```
TENANT="550e8400-e29b-41d4-a716-446655440000"
TOKEN=$(python3 - <<'PY'
import hmac, hashlib, base64, json, time, os
secret = os.environ["HERALD_AUTH_HMAC_SECRET"].encode()
header =
        base64.urlsafe_b64encode(b'{"alg":"HS256","typ":"JWT"}').rstrip(b'
payload = base64.urlsafe_b64encode(json.dumps({
    "tenant": os.environ.get("TENANT", "550e8400-e29b-41d4-
        a716-446655440000"),
    "sub": "ci-bot",
    "exp": int(time.time()) + 3600,
}).encode()).rstrip(b'=')
sig = base64.urlsafe_b64encode(hmac.new(secret,
        header+b'.'+payload,
        hashlib.sha256).digest()).rstrip(b'=')
print((header+b'.'+payload+b'.'+sig).decode())
PY
)
```

Now POST a CloudEvent:

```
curl -sS -X POST \
    -H "Authorization: Bearer $TOKEN" \
    -H "Content-Type: application/cloudevents+json" \
    --data '{
      "specversion": "1.0",
      "id": "01923456-789a-7bcd-abcd-ef0123456789",
      "source": "//your-ci/job/42",
      "type": "digital.vasic.herald.ci.failed",
      "datacontenttype": "application/json",
      "data": {"job": "test-suite", "exit_code": 1, "branch":
        "main"}
    }' \
    http://127.0.0.1:24791/v1/events
```

Expected response: `202 Accepted` + a Receipt JSON body:

```
{
  "event_id": "01923456-789a-7bcd-abcd-ef0123456789",
  "idempotency_key": "01923456-789a-7bcd-abcd-ef0123456789",
  "accepted_at": "2026-05-22T...",
  "recipients": <N>,
  "results": [
    {"channel_id": "tgram", "channel_user_id": "...",
        "evidence": "routed", "channel_msg_id": "..."}
```

```
  ],
  "was_replay": false,
  "outbound_evidence_ids": ["..."]
}
```

If you don't have any subscribers configured for that tenant, `recipients: 0` is correct — the event is still recorded in `events_processed` (so re-POSTing the same idempotency_key returns the cached Receipt with `was_replay: true` and `X-Herald-Replay: true` header).

To get real fan-out: insert a subscriber row (under the tenant) and a `subscriber_aliases` row pointing at a channel + channel_user_id:

```
-- Inside psql, with `SET LOCAL app.current_tenant_id =
        '550e8400-...';` first
INSERT INTO subscribers (tenant_id, handle, display_name)
  VALUES ('550e8400-e29b-41d4-a716-446655440000', 'alice',
        'Alice');

INSERT INTO subscriber_aliases (subscriber_id, channel,
        channel_user_id)
  VALUES ((SELECT id FROM subscribers WHERE handle = 'alice'),
        'tgram', '<your-chat-id>');
```

Now POST the same event again — `recipients: 1`, and the message lands in your Telegram chat. The `outbound_delivery_evidence` row holds the bot-side `message_id` for the audit trail.

---

§9. Verifying delivery (the anti-bluff battery)

Herald's anti-bluff covenant (§107 / spec §11.4) requires every claim to carry **positive runtime evidence**. The battery:

```
# Inheritance + paired mutations (every gate has its own mutation
        proving it catches what it claims):
bash tests/test_constitution_inheritance.sh        # 15/15 PASS
bash tests/test_constitution_inheritance_meta.sh   # META-PASS
bash tests/test_i6_refinement_meta.sh              # 3/3 PASS
bash tests/test_i8_usability_meta.sh               # 5/5 PASS
bash tests/test_wave2_mutation_meta.sh             # 4/4 PASS
bash tests/test_wave3_mutation_meta.sh             # 3/3 PASS
        (M2/M3/M4 SKIP if PG absent)

# Audits:
bash scripts/audit_antibluff.sh                    # 16 PASS / 0
        FAIL / 1 SKIP (telebot 3rd-party SDK)
bash scripts/codegraph_validate.sh                 # 7 PASS / 0
        FAIL / 2 SKIP (HRD-091 codegraph submodule traversal)
```

```
# End-to-end (47 invariants; some SKIP when PG/Redis/Telegram
        aren't reachable):
bash scripts/e2e_bluff_hunt.sh
        # 31+ PASS / 0 FAIL / SKIP-with-reason for absent prereqs
```

ALL must be green. A FAIL is a critical defect; a SKIP must have an explicit `§11.4.3` reason naming what was absent (no PG, no token, no claude binary, etc.).

If your consuming project wires Herald into its own CI:

1. Provision PG + Redis containers in the CI runner (or use the quickstart compose).
2. Export `HERALD_AUTH_MODE=hmac`, `HERALD_AUTH_HMAC_SECRET=ci-secret-32-bytes...`, `HERALD_PG_DSN=...`, `HERALD_REDIS_URL=...`
3. Optionally export `HERALD_TGRAM_BOT_TOKEN` + `HERALD_TGRAM_CHAT_ID` for the live-channel invariants (else they SKIP — fine).
4. Run `bash scripts/e2e_bluff_hunt.sh` — expect ≥45 PASS / 0 FAIL with the containers up.

Herald itself has no CI yet (HRD-???; tracked under operator priorities) — the operator runs the battery locally before each merge.

---

§10. The other flavors (sherald, cherald, iherald, bherald, rherald, scherald)

Each flavor has its own `cmd/<flavor>/main.go` consuming `commons/cli/`. They're invoked the same way (`<flavor> serve --http-port <port>` for serving flavors, `<flavor> <subcommand>` for CLI-only). Per spec §18.0:

Flavor	Default port	Live routes (Wave 3a-b state)	§43 stubs
pherald	24791	`POST /v1/events` (live — Wave 3b)	6 (HRD-029/030/043/044/049/053)
sherald	24793	`GET /v1/safety_state` (live — Wave 3a)	5 (HRD-033/034/040/046/056)
cherald	24792	`GET /v1/compliance` (live — Wave 3a)	11 (HRD-036..039,042,048,050..052,054,055)
iherald	24794	`POST /v1/webhooks/page` (501 stub — HRD-024 pending)	0
bherald	n/a	(CLI-only)	3 (HRD-035/041/045)
rherald	n/a	(CLI-only)	3 (HRD-031/032/045)
scherald	n/a	(CLI-only)	1 (HRD-047)

§43 stubs return non-zero with an HRD pointer in stderr — they're honest 501-equivalents. A consuming project that scripts `pherald commit-push` will hit a non-zero exit until HRD-029 lands. Plan accordingly.

---

§11. Anti-bluff covenant (§107) — what a consumer MUST honor

Herald inherits the Helix Constitution §11.4 / §107 anti-bluff covenant — and any consuming project does too. The verbatim operator mandate:

> "all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well (if not there already)!"

Practical implications for a consuming project:

1. **Every test asserts positive runtime evidence.** Not "no error returned", not "config file exists", not "binary compiled". Real HTTP response → real PG row → real channel API reply.
2. **Every gate has a paired mutation.** If your gate asserts property X, you MUST also have a test that mutates code to remove X and verifies the gate FAILs. A gate without a paired mutation is itself a §11.4 PASS-bluff.
3. **§107 anchor in every root doc.** Your project's `CLAUDE.md`, `AGENTS.md`, `Constitution.md`, `QWEN.md` MUST contain the verbatim mandate text. Herald's inheritance gate I8a/b/c enforces this for Herald's own root docs; the same applies recursively to submodules. Use the verbatim text from `<discovered>/Constitution.md`.
4. **SKIP-with-reason, never silent skip.** When a test can't run (no container runtime, no operator creds, no test fixture), it MUST emit a `SKIP` line citing an explicit closed-set reason: `hardware_not_present`, `credentials_not_present`, etc. A silent skip is a bluff.
5. **Multi-mirror push contract.** Every commit touching Herald-tracked files MUST land on all four mirrors atomically (`origin` is configured as fan-out URL list — one `git push origin main` does it). A single-mirror push is a §11.4 violation.

---

§12. Known limitations + Wave 4 transport roadmap

Wave 3b state (current): the §32 Runner is live, JWT auth is in place, idempotency works, real Postgres-backed evidence rows are written. What's not yet there:

- **§43 command bodies (28 HRDs).** `<flavor> <command>` returns non-zero with HRD pointer for ~all of them. Wave 4+ pursues these mechanically.
- **Constitution bindings (HRD-018..027).** The `Evaluator` interface + Registry are wired; flavor-specific evaluators (creds-scan, destructive-op detection, force-push gate, etc.) are not yet written. Wave 5+.
- **iherald** `/v1/webhooks/page` still 501. HRD-024 pending.
- **Channel adapters beyond null:// + Telegram.** Slack, Email, Max, Discord, Teams, Lark, WhatsApp, Viber, ntfy, Gotify, Webhook are all "reserved env var names" but no code. Each is its own ~5-task workstream when needed.

- **Subscriber preference filtering** is permissive in Wave 3b — every subscriber receives every event. CategoryPref / WorkflowPref / QuietHours (per spec §7.2-§7.3) is a future HRD.

Wave 4 transport upgrade (designed but not yet implemented):

Per the operator mandate captured 2026-05-22 + the design doc at `docs/superpowers/specs/2026-05-22-http3-brotli-toon-design.md`:

- **HTTP/3 (QUIC) primary** — every REST surface must speak HTTP/3 over UDP as the default transport.
- **HTTP/2 fallback** — for clients that can't speak QUIC. Dual TCP+UDP listener inside the binary; `Alt-Svc` header advertises the HTTP/3 endpoint to capable clients.
- **Brotli compression default** — `Content-Encoding: br` with fallback to gzip / identity per `Accept-Encoding`.
- **TOON primary data type** — Token-Oriented Object Notation, a JSON variant optimized for LLM token efficiency (30-60% savings). `Content-Type: application/toon` preferred; `application/json` fallback.
- **2-wave split**: Wave 4a (HTTP/3 + Brotli, ~10 tasks) and Wave 4b (TOON, ~7 tasks).

When Wave 4a + 4b land, this guide will gain a new transport-negotiation section. Until then, consumers connect over HTTP/2 with JSON exactly as documented above.

---

§13. Multi-mirror push contract

Herald lives on **four mirrors** simultaneously:

Host	URL
GitHub	`git@github.com:vasic-digital/Herald.git`
GitLab	`git@gitlab.com:vasic-digital/herald.git`
GitFlic	`git@gitflic.ru:vasic-digital/herald.git`
GitVerse	`git@gitverse.ru:vasic-digital/Herald.git`

The `origin` remote is configured as a push-URL fan-out — one `git push origin main` propagates to all four atomically. If you clone Herald (as a submodule or standalone), `git remote -v` will show the fan-out as a single push-URL list under `origin`.

For consumers who **only need read access**, any single mirror works — clone from whichever you prefer.

---

§14. Where to ask for help

- **Operator credentials guide**: `docs/guides/OPERATOR_CREDENTIALS.md` — full reference for every supported messenger + dispatcher.
- **Per-messenger guides**: `docs/guides/messengers/{TELEGRAM,SLACK,EMAIL,MAX,DISCORD,TEAMS,LARK,WHATSAPP,VIBER}.md` (some are stubs for not-yet-implemented channels — code-complete: Telegram).
- **Per-dispatcher guides**: `docs/guides/dispatchers/{CLAUDE_CODE,ANTHROPIC,OPENCODE,AIDER,GEMINI,CURSOR}.md` (code-complete: CLAUDE_CODE).

- **Herald Constitution**: `docs/guides/HERALD_CONSTITUTION.md` — project-specific constitutional extensions on top of the inherited Helix Constitution.
- **Parent-discovery details**: `docs/guides/CONSTITUTION_INHERITANCE.md`.
- **Spec V3** (the source of truth for the API + data model): `docs/specs/mvp/specification.V3.md`.
- **Open work**: `docs/Issues.md`. Closed work: `docs/Fixed.md`. Running status: `docs/Status.md`.

If something in this guide is wrong or out of date, the spec wins. If the spec is unclear, file an HRD in `docs/Issues.md` per spec V3 §8.3.

Quick checklist for a fresh integration

- [ ] `git submodule add` Herald + its referenced submodules
- [ ] `go work init && go work use ./...` inside Herald
- [ ] `go build ./pherald/cmd/pherald` succeeds clean
- [ ] Quickstart compose stack up (`podman-compose up -d`) OR your own PG + Redis reachable
- [ ] `pherald migrate up` → `schema version: 12`
- [ ] `pherald wizard credentials telegram` (or set env vars manually)
- [ ] Generate a JWT (HMAC or via your IdP)
- [ ] `pherald serve --http-port 24791` runs without errors
- [ ] `curl -X POST http://127.0.0.1:24791/v1/events` with the JWT returns 202 + Receipt
- [ ] `bash scripts/e2e_bluff_hunt.sh` shows ≥31 PASS / 0 FAIL
- [ ] Subscribers + aliases inserted for your tenant; second POST shows `recipients > 0`
- [ ] Real Telegram delivery observed (operator's chat receives the message)
- [ ] `outbound_delivery_evidence` row count matches the number of dispatches
- [ ] Multi-mirror push works from the consuming project's local clone (if you have write access)